

HOMESERVANT

Frontend Developer Guide

How the HomeServant Flutter app is built

Version of 27 September 2026. Written from the code in the HomeServant-MVP repository.

Source: `docs/guides/frontend_guide.html` — rebuild with `python3 docs/guides/build.py`.

Contents

1. What the app is and what it runs on
2. Folder layout
3. How the app starts
4. Navigation and route guards
5. State: `AppState`
6. Talking to the backend
7. Real-time updates, sounds and browser pop-ups
8. The screens, area by area
9. Look and feel: themes, colors and the contrast rule
0. Web-specific setup (Vercel, security policy, sessions)
1. Testing, building and conventions

1. What the app is and what it runs on

HomeServant is one Flutter app that serves four audiences:

- **Tenants**, who find and rent properties;
- **Landlords**, who list and manage properties and bookings;
- **Marketplace customers and vendors** (a tenant can also open a shop);
- **Admins**, through the admin console.

The same code builds the website (hosted on Vercel) and the Android/iOS apps. All data lives on the HomeServant API (see the Backend Developer Guide). The app holds no business data of its own beyond a few per-device preferences.

Main packages and why they're used

Package	Why
<code>provider</code>	Makes the single <code>AppState</code> object available to every screen. It's simple and well understood, which suits a small team.
<code>go_router</code>	URL-based routing for the entry, auth, dashboard and admin screens, so web addresses and the browser back button work, plus a central place for sign-in redirects.
<code>dio</code>	HTTP client. Its interceptors let one place attach the login token and refresh it when it expires.
<code>socket_io_client</code>	Live updates (new messages, notifications, claims) over one WebSocket.
<code>flutter_secure_storage</code>	Keeps login tokens in the phone's keychain.
<code>shared_preferences</code>	Per-device preferences such as the dashboard theme and notification toggles.
<code>google_sign_in</code> / <code>_web</code>	"Continue with Google": the native picker on phones, Google's own rendered button on web.
<code>image_picker</code> , <code>file_picker</code> , <code>mime</code>	Choosing photos and documents to upload, and detecting their type.
<code>url_launcher</code>	Opens Paystack's checkout page for payments.
<code>pdf</code> , <code>printing</code>	Generating PDFs in the app (tenancy agreements, chat transcripts).
<code>audioplayers</code>	The admin alert sound.
<code>video_player</code> , <code>flutter_svg</code> , <code>device_info_plus</code>	Splash video and listing walkthrough videos; SVG icons; the device name recorded at login.

2. Folder layout

Folder	What's in it
<code>lib/main.dart</code> , <code>lib/app.dart</code>	Entry point and the top-level <code>MaterialApp.router</code> .
<code>lib/routes/app_router.dart</code>	Every <code>go_router</code> route and the redirect rules (<code>appRedirect</code>).
<code>lib/state/app_state.dart</code>	The single app-wide state object (section 5).
<code>lib/api/</code>	<code>ApiClient</code> (HTTP plus token refresh), one <i>repository</i> per backend area, <code>TokenStorage</code> , <code>api_config.dart</code> .
<code>lib/api/models/</code>	Data classes that mirror API responses, each with a <code>fromApi</code> parser.
<code>lib/features/<area>/</code>	Screens grouped by audience: <code>splash</code> , <code>onboarding</code> , <code>auth</code> , <code>dashboard</code> (tenant and shared), <code>landlord</code> , <code>Market place</code> , <code>admin</code> .
<code>lib/widgets/</code>	Reusable pieces: buttons, fields, bottom sheets, gates, the notification banner, the bank details screen, the Google button, and more.
<code>lib/services/</code>	Anything that isn't a screen or an API call: live connection (<code>ChatSocketService</code>), alert sound, browser pop-ups, Google sign-in, app icon switching.
<code>lib/core/</code>	Colors, text styles, the Material theme, date and number formatting, responsive helpers.
<code>lib/models/</code>	<code>UserRole</code> and <code>DashboardTheme</code> , which drive the colors of each area.
<code>test/</code>	Navigation tests, route-guard unit tests, model parsing tests, App Lock tests.
<code>web/</code> , <code>vercel.json</code>	Web entry HTML, icons, and the Vercel build and security settings.

3. How the app starts

1. `main()` calls `assertSecureApiBaseUrl()`, which refuses to run a release build pointed at a non-HTTPS API, then starts `HomeServantApp`.
2. `HomeServantApp` creates the one `AppState` and the router, and calls `AppState.load()`. That restores device preferences and, if a login token is saved, fetches the user's profile (`GET /users/me`) and their starting data. Until it finishes, `isLoading` is false and the router doesn't redirect anywhere.
3. `MaterialApp.router` wraps every page in three layers, outermost first:
 - `SessionExpiredGate` : shows "you were signed out" when the refresh token dies.
 - `AppLockGate` : the optional PIN lock on phones.
 - `NotificationBannerOverlay` : the in-app notification banner, the admin alert sound and browser pop-ups.

Why one app instead of separate apps? Tenants, landlords, vendors and admins share most building blocks: sign-in, chat, notifications, uploads, payments. One codebase means one fix applies everywhere.

4. Navigation and route guards

Named routes (go_router)

Route	Screen
/	Splash screen on phones; the marketing site (<code>WebLandingScreen</code>) on web.
/get-started , /login , /signup	Choice screens, all built on the shared <code>TwoChoiceScreen</code> layout.
/login-tenant , /login-landlord	<code>LoginRoleScreen</code> : email and password, Google, forgot password.
/signup-tenant , /signup-landlord	<code>SignupRoleScreen</code> : email, password, Google.
/verify-otp , /login-2fa	<code>VerifyOtpScreen</code> : the signup code, or the two-factor code at login.
/signup-tenant-1 , /signup-landlord-1	<code>SignupBasicsScreen(role)</code> : name, phone, date of birth, plus role extras.
/signup-tenant-2 , /signup-landlord-2	<code>SignupDetailsScreen(role)</code> : photo, ID, gender, occupation, marital status.
/forgot-password , /reset-password	Password reset by emailed code.
/app-lock-verify	PIN entry after login, when App Lock is on.
/dashboard	<code>TenantDashboardScreen</code> or <code>LandlordDashboardScreen</code> , depending on the role.
/marketplace	<code>MarketplaceNavigatorHost</code> : the marketplace's own nested navigator.
/admin-login , /admin	Admin sign-in and <code>AdminShell</code> (the console).

Everything past the dashboards (property details, chats, settings, marketplace screens, admin detail pages) opens with plain `Navigator.push` . Those are drill-down pages that don't need their own web address.

Why the marketplace has its own route and navigator: it has many screens of its own. Giving it one route means a single browser *Back* press from anywhere inside it returns to the dashboard, instead of skipping past the dashboard to an older page.

Redirect rules (`appRedirect`)

The router asks `appRedirect(appState, location)` on every navigation and whenever `AppState` changes. It's a plain function, so it's unit-tested in `test/router_redirect_test.dart` .

1. **Signed out** on a protected page: sent to `/get-started` . Pre-auth pages (login, signup, code entry) are allowed.
2. **Signed in** on an entry page (`/` , `/get-started` , `/login` , `/signup`): sent to `/dashboard` , or `/admin` for admins.

The login and code screens themselves are deliberately *not* redirected. They become "signed in" partway through their own flow, and need to run their own next step, such as the App Lock

PIN or finishing profile setup.

3. **Admins** never land on `/dashboard` ; they're sent to `/admin` .
4. **Unfinished profiles:** a tenant or landlord whose server-side `profileCompleted` flag is false is sent from `/dashboard` or `/marketplace` back into the signup steps. Name and phone decide whether that's step 1 or step 2.

This relies only on the flag the server sends with every login response. Checking individual profile fields instead used to send finished users back into setup on every login.

5. State: `AppState`

`lib/state/app_state.dart` is a single `ChangeNotifier`, provided at the top of the app. Screens read it with `context.watch<AppState>()` (rebuild on change) or `context.read<AppState>()` (one-off calls).

Section	Holds
Session and profile	<code>userId</code> , <code>role</code> , <code>name</code> , <code>phone</code> , <code>address</code> , <code>photo</code> , <code>gender</code> , <code>occupation</code> , <code>marital status</code> , <code>profileCompleted</code> , 2FA flag, bank details.
Auth	<code>signup</code> , <code>verifySignupOtp</code> , <code>login</code> and <code>loginWithGoogle</code> (which return a <code>LoginOutcome</code> : success, two-factor needed, reactivation needed, or email verification needed), <code>completeProfile</code> , <code>logout</code> , <code>deactivate</code> and <code>delete</code> .
Notifications	The list, unread count, mark read.
Properties, wishlist, bookings	The public feed, a landlord's own listings, favorites, the tenant's and the landlord's bookings.
Vendor	Whether this user has a shop.
Admin	<code>adminLevel</code> , <code>adminOnDuty</code> (server-side), and the device-only <code>adminAlertSound</code> and <code>adminBrowserNotifications</code> .
Device preferences	Dashboard theme, notification toggles, banner auto-dismiss, App Lock.

Repositories are exposed on `AppState` (for example `appState.chat`, `appState.admin`, `appState.uploads`). Screens that manage their own lists, such as chats or admin tables, call them directly rather than storing everything in `AppState`.

Where it's saved: login tokens go to the phone's secure keychain, or to the browser tab's `sessionStorage` on web (section 10). Device preferences go to `SharedPreferences`, or `sessionStorage` on web. Profile data is never saved locally; it's re-fetched from the API at start-up.

Why one state object? The app's shared data is small and closely linked (the signed-in user drives almost everything). One well-organised object is easier for a small team than a store per feature. If it grows much further, splitting it by area is the natural next step.

6. Talking to the backend

Where the API is

`lib/api/api_config.dart` reads `API_BASE_URL` from the build command (`--dart-define=API_BASE_URL=https://.../api`). The Vercel build sets it to the Render server.

ApiClient : one HTTP client for everything

- Adds `Authorization: Bearer <access token>` to every request, except auth calls marked `skipAuth` .
- On a `401` , calls `POST /auth/refresh` once, saves the new token pair, and retries the request. Several requests failing together share one refresh, because the server's refresh tokens can only be used once. If refreshing fails, `onSessionExpired` signs the app out and shows the "session expired" message.
- `call(...)` turns every network error into an `ApiException` carrying the server's own message, so screens simply show `e.message` .

Repositories: one per backend area

Repository	Backend area it calls
<code>AuthRepository</code>	<code>/auth/*</code> : signup, verify, login, Google, 2FA, refresh, logout, password reset and change, deactivate, delete.
<code>UsersRepository</code>	<code>/users/me</code> , <code>/users/me/bank-details</code> .
<code>PropertiesRepository</code>	<code>/properties</code> .
<code>BookingsRepository</code>	<code>/bookings</code> : create, pay, inspection, move-in, refund, renew, landlord responses, tenancy agreement.
<code>FavoritesRepository</code> , <code>ReviewsRepository</code>	<code>/favorites</code> , <code>/reviews</code> .
<code>ChatRepository</code>	<code>/threads</code> : inbox, messages, support chat, claim, resolve, transfer, reassign, summary.
<code>NotificationsRepository</code>	<code>/notifications</code> .
<code>VendorsRepository</code> , <code>MarketplaceProductsRepository</code> , <code>MarketplaceOrdersRepository</code>	<code>/vendors</code> , <code>/marketplace/products</code> , <code>/marketplace/orders</code> .
<code>PaystackRepository</code>	<code>/paystack/banks</code> and account-name lookup for bank details.
<code>UploadsRepository</code>	<code>/uploads/sign</code> , then uploads straight to Supabase Storage.
<code>AdminRepository</code>	<code>/admin/*</code> and <code>/reports/*</code> for the console.

A typical request, end to end

Screen button tap

```
-> context.read<AppState>().someAction()      (or a repository directly)
-> XRepository.method() -> ApiClient.call(() => dio.post('/path', data: {...}))
-> server responds with JSON
-> Model.fromApi(json) -> AppState updates fields + notifyListeners()
-> screens using context.watch rebuild
```

Uploading photos and files

1. The user picks a file (`upload_picker.dart`).
2. `UploadsRepository.upload` reads its bytes and works out its type from the content. On web the picked file is a `blob:` address, which is why the security policy allows `blob:` .
3. It asks the API for a one-time upload link (`POST /uploads/sign`) and sends the bytes straight to Supabase Storage with `PUT` .
4. It saves the returned public URL on the record, for example `PATCH /users/me {profilePhotoUrl}` . The server checks that the URL is really one of its own images before accepting it.

Payments

Booking a property or buying a product returns a Paystack `authorizationUrl` , which the app opens with `url_launcher` . The server learns that the payment succeeded from Paystack directly, via its webhook, and the app picks up the new status on its next refresh or live notification.

Google sign-in

On phones, our own button opens the native account picker. On web, Google's own rendered button is shown, because on web that's the only way to get the ID token the server verifies. Both end in `AppState.loginWithGoogle` .

7. Real-time updates, sounds and browser pop-ups

`ChatSocketService` opens one `Socket.IO` connection after login, authenticated with the access token, and turns server events into Dart streams.

Event	What the app does
<code>message:new</code>	Open chat screens append the message; inbox lists reload.
<code>message:read</code>	Your sent messages show "Seen".
<code>notification:new</code>	Banner, list refresh, admin sound and browser pop-up, unless the push is marked <code>silent</code> .
<code>thread:claimed</code>	Admin queues reload, the matching banner is dismissed, and open chats re-check whether you can still reply.
<code>thread:access-revoked</code>	An open chat switches to read-only with a notice.
<code>admin:badges-changed</code>	The admin console reloads its badge counts.

Admin-only extras, all in `NotificationBannerOverlay`:

- **Alert sound** (`ChatSoundService`): at most one sound per 10 seconds, only if the device's sound setting is on.
- **Web Push** (everyone, web only): `offerBrowserNotifications` offers to turn notifications on once per page load on each dashboard. `AppState.enableWebPush` asks the browser from that tap, registers `web/push-sw.js` (scope `./push/`, so it never touches Flutter's own files) and sends the subscription to the backend; `syncWebPush` re-registers after every sign-in, and sign-out unsubscribes this browser. The worker skips showing a push while the app is open and in view, and uses the same tag as the in-tab pop-up so they never double up.
- **Browser pop-ups** (`services/browser_notifications*.dart`): shown only when the console tab isn't in view. They use the browser's own Notification feature, through a web-only file loaded with a conditional import. It does nothing on phones.

8. The screens, area by area

Entry and sign-in (`features/splash` , `onboarding` , `auth`)

Screen	Purpose	Main API calls
SplashScreen / WebLandingScreen	First screen on phones / marketing site on web.	none
GetStarted, Login, Signup	Choose Login or Sign Up, then the role.	none
LoginRoleScreen	Email and password login; Google; forgot password.	<code>/auth/login</code> , <code>/auth/google</code>
SignupRoleScreen	Create account (email and password) or Google.	<code>/auth/signup</code>
VerifyOtpScreen	Signup code or login 2FA code; resend.	<code>/auth/verify-signup</code> , <code>/auth/verify-2fa</code> , <code>/auth/resend-otp</code>
SignupBasicsScreen, SignupDetailsScreen	Profile setup steps 1 and 2.	<code>PATCH /users/me</code> , uploads
Forgot / Reset password	Reset by emailed code.	<code>/auth/forgot-password</code> , <code>/auth/reset-password</code>

Tenant (`features/dashboard`)

Screen	Purpose	Main API calls
TenantDashboardScreen	Property feed with search and filters, location header, messages and notification icons.	<code>GET /properties</code>
PropertyDetailScreen	Gallery, video, details. Tenants: Rent/Book, favorite, and Message Landlord once they have paid (a paid booking on this property). <code>ChatThreadScreen</code> reads the thread summary's <code>lockedReason</code> and shows it instead of the composer when payment rules close a tenant/landlord chat (for example after a refund). <code>SYSTEM</code> messages render as a centred notice rather than a bubble. Owners: Edit Property and Delete Listing (confirmed first; the server's reason is shown if deletion is refused).	<code>/properties/:id</code> , <code>/bookings</code> , <code>/threads</code>
MessagesScreen, ChatThreadScreen	Inbox (optionally marketplace-only) and a single chat, with photos and PDF export.	<code>/threads</code> , <code>/threads/:id/messages</code>
NotificationsScreen (+ detail)	Notifications; chat ones show the conversation's status and Reply.	<code>/notifications</code> , <code>/threads/:id/summary</code>

History, Wishlist	Bookings and payments; saved properties.	<code>/bookings/mine</code> , <code>/favorites</code>
Profile, EditProfile, Settings	Profile and photo, theme, notifications, 2FA, App Lock, deactivate or delete.	<code>/users/me</code> , <code>/auth/*</code>

Landlord (`features/landlord`)

Screen	Purpose	Main API calls
LandlordDashboardScreen	Tabs: Home, Messages, Bookings, Profile.	(tabs below)
LandlordHomeTab	Stats (including the real booking-request count), an Upload a Property button, incoming bookings, uploads.	<code>/bookings/landlord</code>
LandlordEvictionPanel (<code>widgets/eviction_widgets.dart</code>)	On the owner's property page: current tenant, Request eviction, the pending request and Withdraw. <code>TenantEvictionNotice</code> shows it on the tenant's History card with Respond.	<code>/evictions/*</code>
LandlordPropertiesScreen, LandlordAddPropertyScreen	Own listings; add or edit with photos and video.	<code>/properties</code> , uploads
LandlordBookingsScreen	Requests, inspections, move-ins, rejections.	<code>/bookings/*</code>
LandlordMessagesScreen	Inbox with All and Unread filters, plus search.	<code>/threads</code>
BankDetailsScreen.landlord()	Payout account, checked with Paystack.	<code>/paystack/*</code> , <code>/users/me/bank-details</code>

Marketplace (`features/Market place`)

Screen	Purpose	Main API calls
MarketplaceAuthScreen	Shop as a customer, become a vendor, or open your shop.	<code>/vendors/me</code>
MarketplaceHomeScreen, ProductDetail	Browse, cart and checkout (opens Paystack).	<code>/marketplace/products</code> , <code>/marketplace/orders</code>
OrderHistoryScreen	Your orders; confirm received; message the vendor.	<code>/marketplace/orders/mine</code>
VendorSignup, VendorDashboard, VendorProducts, AddProduct	Apply for a shop; manage products and orders.	<code>/vendors</code> , <code>/marketplace/*</code>
VendorEditProfile, BankDetailsScreen.vendor()	Shop profile and payout account.	PATCH <code>/vendors/me</code>

Admin console (`features/admin`)

`AdminShell` hosts the tabs: Dashboard, Users, Vendors, Reports, and a More sheet with Properties, Marketplace, Messages, Admins, Activity Log and Chat Log. Detail pages cover a user, vendor, property, order or report. What each admin level can see is described in the Admin Guide. The shell also:

- enforces the 15-minute idle sign-out;
- forces a password change when `mustChangePassword` is set;
- loads badge counts live;
- hosts Account Settings: 2FA, change password, On duty, alert sounds, browser pop-ups.

9. Look and feel

- **Fonts:** Quity for headings, Givonic for body text. Both are bundled with the app, never downloaded (`AppTextStyles`). Neither has `#`, dashes, bullets, ellipsis or arrows, so a bundled subset of DejaVu Sans (`HSFallback` , `assets/fonts/`) is set as `fontFamilyFallback` on every style and on the theme. Without it the browser fetches a fallback font from Google at runtime and those characters render blank if that fails. When you add a new symbol to the UI, check it's covered, or re-run `pyftsubset` with the extra range.
- **Fixed brand colors:** `AppColors` (navy, gold, sand and so on). The auth screens and admin console use these directly.
- **Role colors:** `UserRole` gives the auth screens navy for tenants and gold/sand for landlords.
- **Dashboard themes:** `DashboardTheme` offers user-selectable palettes (Classic, Midnight, Sand) for the dashboards, marketplace and vendor screens. Each theme defines matching pairs: `background/foreground` , `surface/onSurface` , `accent/onAccent` .

The contrast rule (from `CLAUDE.md`): every piece of text needs an explicit color that is checked against the exact background it sits on. Only use each theme pair together; `background/foreground` flip between themes. When giving a field a custom fill color, always pass its text color too. Invisible text has been this app's most common bug.

Layouts are designed for phones first and centred with a maximum width on larger screens (`ResponsiveCenter`).

10. Web-specific setup

Vercel build

`vercel.json` clones Flutter, runs `flutter build web --release --dart-define=API_BASE_URL=...`, and serves `build/web`. Every pull request gets a preview deployment.

Content-Security-Policy

The site sends a strict security policy. Anything the app loads from elsewhere must be listed in `vercel.json`; if something silently stops working on web, check here first.

Allowed	Why
<code>https://www.gstatic.com</code>	Flutter's rendering engine (CanvasKit).
<code>https://fonts.gstatic.com</code>	Flutter's fallback fonts. Without it, text in default fonts rendered invisible.
<code>blob: (connect)</code>	Reading a picked photo before upload. Without it, uploads failed with "could not reach the server".
<code>https://*.supabase.co</code>	Uploading to and showing images from storage.
API host (<code>https</code> and <code>wss</code>)	The REST API and the live connection.
<code>https://accounts.google.com</code>	Google sign-in button and popup.

Sessions per tab

On web, login tokens and preferences are kept in `sessionStorage`, which is separate for each tab. That way two people signed in on different tabs of the same browser never share a session. The trade-off is that opening a new tab means signing in again.

Web addresses

The app uses hash-style addresses (`/#/admin-login`), so any page can be refreshed without extra server setup.

11. Testing, building and conventions

```
flutter pub get
flutter analyze      # must report "No issues found!"
flutter test         # all tests must pass
flutter run -d chrome --dart-define=API_BASE_URL=http://localhost:3000/api
flutter build web --release --dart-define=API_BASE_URL=https://<api-host>/api
```

Conventions

- **New backend call:** add a method to the relevant repository, wrap it in `_client.call`, and parse the response with a model's `fromApi`.
- **New named page:** add a `GoRoute` and decide whether it's a pre-auth page (`_preAuthPaths`). Add a test to `router_redirect_test.dart` if it affects redirects.
- **Before copying a screen:** reuse the shared widgets in `lib/widgets`. Parameterise by role instead of duplicating; earlier copies drifted apart and caused bugs.
- **Web-only code:** put it behind a conditional import with a no-op stub, as in `browser_notifications*.dart` and `google_web_button*.dart`.
- **Permissions:** never rely on hiding a button. The server must refuse the action too.